ETAPA A · ENTENDER · SPRINT 3 DE 17

Apuntes del Sprint 3 · Del dato al servicio: SQL, NoSQL y AutoML

Las diapositivas del Sprint 3 son esquemáticas y llenas de código. Estos apuntes **explican con calma qué hace cada pieza**, con una mini-base de datos de ejemplo cuyos resultados puedes comprobar a mano. Léelos antes de abrir DB Browser, o cuando una consulta te dé un error que no entiendes. No hace falta saber programar. **SQL no es programar: es hacer preguntas a una tabla casi en inglés.**

0. De qué va el sprint (en 1 minuto)

El sprint recorre el camino que hace un dato hasta convertirse en un servicio:

```
Datos guardados (SQL / NoSQL)  →  Primer modelo (AutoML)  →  Servicio que otros pueden usar (API)
```

Parte	Pregunta	Herramienta
S03_01	¿Cómo guardo y pregunto a los datos?	SQL en DB Browser
S03_02	¿Y si los datos no caben bien en tablas?	NoSQL (MongoDB)
S03_03	¿Cómo consigo un primer modelo rápido y lo pongo a funcionar?	AutoML y MLOps

El reto: el «primer modelo en producción»: usar los datos de TurisData para predecir cancelaciones con AutoML, elegir el umbral por dinero, publicarlo como servicio y **cerrar y borrar todo lo creado** (coste 0 €).

DÍA 1 · Bases de datos relacionales y SQL

1. Qué es una base de datos relacional

Es un conjunto de **tablas conectadas entre sí**, como varias hojas de cálculo donde unas apuntan a otras. En TurisData hay tres:

Tabla	Una fila es...	Clave primaria
`alojamientos`	un hotel, apartamento o casa rural	`id_alojamiento`
`clientes`	una persona (email anonimizado)	`id_cliente`
`reservas`	una reserva de un cliente en un alojamiento	`id_reserva`

- **Clave primaria (PK):** identifica **cada fila de forma única** (como el DNI).
- **Clave foránea (FK):** una columna que **apunta a la clave primaria de otra tabla**. En `reservas`, `id_alojamiento` apunta a `alojamientos`.

¿Por qué separar en tablas? Para no repetir datos. Si en cada reserva escribieras a mano «Reino Unido», tendrías «UK», «uk» y «Reino Unido» para lo mismo. Con una tabla de países y una clave foránea, **el dato se escribe una sola vez**.

> Los datos son **sintéticos**: sirven para practicar, no son estadísticas reales del turismo en Canarias.

Mini-base de datos para los ejemplos

Usaremos esta versión pequeña (así puedes comprobar a mano cada resultado):

alojamientos

id_alojamiento	nombre	isla	plazas
1	Hotel Sol	Tenerife	200
2	Casa Palma	La Palma	6
3	Apart Mar	Tenerife	80
4	Hotel Dunas	Gran Canaria	150
5	Villa Norte	Tenerife	12

reservas (`cancelada` vale 1 si se canceló y 0 si no)

id_reserva	id_alojamiento	canal	noches	cancelada
1	1	web	3	0
2	1	web	2	1
3	3	agencia	5	0
4	4	web	4	1
5	4	directo	2	0
6	2	directo	7	0
7	1	agencia	1	1
8	3	web	6	0

2. SELECT: preguntar a una tabla

Una consulta se **lee de arriba abajo como una frase**, pero el ordenador la ejecuta en otro orden:

```
SELECT nombre, plazas            -- 3. qué columnas quiero ver
FROM alojamientos                -- 1. de qué tabla
WHERE isla = 'Tenerife'          -- 2. qué filas (filtro)
  AND plazas > 50
ORDER BY plazas DESC             -- 4. en qué orden
LIMIT 2;                         -- 5. cuántas filas como máximo
```

Resultado con nuestra mini-base:

nombre	plazas
Hotel Sol	200
Apart Mar	80

(Villa Norte está en Tenerife pero tiene 12 plazas, así que el filtro `plazas > 50` la deja fuera.)

Reglas básicas:

- Cada consulta **termina en punto y coma**.

- Los textos van entre **comillas simples**: `'Tenerife'`.
- En DB Browser: selecciona la consulta y pulsa **Ctrl+Intro** para ejecutar solo esa.
- `--` empieza un comentario (el ordenador lo ignora).

3. JOIN: unir tablas

La reserva guarda solo `id_alojamiento` (un número). Para ver **la isla** hay que ir a la otra tabla: eso es un **JOIN**.

```
SELECT r.id_reserva, a.isla, r.canal
FROM reservas r
JOIN alojamientos a ON a.id_alojamiento = r.id_alojamiento;
```

- `r` y `a` son **apodos** (alias) de las tablas, para escribir menos.
- `ON …` dice **qué columna de una tabla coincide con la de la otra** (clave foránea = clave primaria).

La reserva 1 (alojamiento 1) se une con «Hotel Sol», así que obtiene la isla *Tenerife*. Resultado de las 8 reservas: 1→Tenerife, 2→Tenerife, 3→Tenerife, 4→Gran Canaria, 5→Gran Canaria, 6→La Palma, 7→Tenerife, 8→Tenerife.

JOIN frente a LEFT JOIN

Tipo	Qué devuelve
JOIN (interno)	Solo las filas que tienen pareja **en las dos tablas**
LEFT JOIN	**Todas** las de la tabla izquierda; si no tienen pareja, lo de la derecha sale `NULL` (vacío)

Truco para encontrar **lo que NO tiene pareja** (alojamientos sin reservas):

```
SELECT a.nombre
FROM alojamientos a
LEFT JOIN reservas r ON r.id_alojamiento = a.id_alojamiento
WHERE r.id_reserva IS NULL;
```

Resultado: **Villa Norte** (la única que nunca aparece en `reservas`). Se escribe `IS NULL`, **nunca** `= NULL`.

4. GROUP BY y HAVING: resumir por grupos

`GROUP BY` agrupa las filas que tienen el mismo valor y calcula **un resumen por grupo**. Funciones de resumen: `COUNT` (contar), `SUM` (sumar), `AVG` (media), `MIN`, `MAX`.

```
SELECT canal,
       COUNT(*) AS reservas,
       ROUND(100 * AVG(cancelada), 1) AS pct_cancelacion
FROM reservas
GROUP BY canal;
```

Resultado, que puedes comprobar a mano mirando la tabla:

canal	reservas	pct_cancelacion
agencia	2	50.0
directo	2	0.0
web	4	50.0

Truco clave: como `cancelada` vale 0 o 1, su **media es directamente la proporción de cancelaciones**. Para `web`: (0 + 1 + 1 + 0) / 4 = 0,5 → 50 %. Esa es la pregunta del cliente.

WHERE frente a HAVING

	Cuándo actúa	Filtra…
WHERE	**Antes** de agrupar	filas
HAVING	**Después** de agrupar	grupos

```
SELECT canal, COUNT(*) AS n
FROM reservas
GROUP BY canal
HAVING COUNT(*) > 2;     -- solo canales con más de 2 reservas
```

Resultado: solo `web` (4 reservas). Si pones `COUNT(*)` en un `WHERE`, da error: todavía no hay grupos.

5. Subconsultas y CTE

Una **subconsulta** es una consulta **dentro de otra**.

Escalar (devuelve un solo valor): reservas más largas que la media.

```
SELECT * FROM reservas
WHERE noches > (SELECT AVG(noches) FROM reservas);
```

La media de noches es (3+2+5+4+2+7+1+6)/8 = 30/8 = **3,75**, así que salen las reservas de 5, 4, 7 y 6 noches.

Con IN (devuelve una lista de ids): reservas en alojamientos de Tenerife.

```
SELECT * FROM reservas
WHERE id_alojamiento IN (SELECT id_alojamiento FROM alojamientos WHERE isla = 'Tenerife');
```

CTE (`WITH`): pone **nombre a un paso intermedio** para que la consulta se lea de arriba abajo. Canales con más cancelaciones que la media global (la media global es 3/8 = 37,5 %):

```
WITH por_canal AS (
  SELECT canal, AVG(cancelada) AS tasa FROM reservas GROUP BY canal
)
SELECT canal, ROUND(100 * tasa, 1) AS pct
FROM por_canal
WHERE tasa > (SELECT AVG(cancelada) FROM reservas);
```

Resultado: `agencia` (50 %) y `web` (50 %). `directo` (0 %) queda fuera.

6. CRUD y transacciones: modificar datos con cuidado

CRUD son las cuatro operaciones básicas:

Letra	Sentencia	Qué hace
Create	`INSERT INTO … VALUES (…)`	añadir una fila
Read	`SELECT`	leer
Update	`UPDATE … SET … WHERE …`	cambiar filas
Delete	`DELETE FROM … WHERE …`	borrar filas

> **La regla más importante del sprint: sin `WHERE`, `UPDATE` y `DELETE` afectan a TODA la tabla.**

Imagina `UPDATE reservas SET precio_noche = 135;` sin `WHERE`: **todas las reservas** pasan a costar 135. Para evitarlo:

1. **Primero un** `SELECT` con el `WHERE` que vas a usar, para ver qué filas tocarías.
2. **Después** el `UPDATE` con **el mismo** `WHERE`.
3. **Practica siempre en una copia** (`reservas_practica`), **nunca en la tabla original**.

Una **transacción** agrupa cambios para poder **deshacerlos**:

```
BEGIN;                                -- empiezo
DELETE FROM reservas_practica WHERE canal = 'agencia';
ROLLBACK;                             -- ¡me arrepiento! todo vuelve a como estaba
-- con COMMIT; se confirmaría y ya no habría vuelta atrás
```

7. Modelado, restricciones e índices

Restricciones: reglas que hacen que **el dato malo falle al entrar**, en vez de descubrirlo meses después.

```
CREATE TABLE p_reservas (
  id_reserva   INTEGER PRIMARY KEY,               -- único
  pais         TEXT REFERENCES p_paises(codigo),   -- clave foránea: debe existir
  noches       INTEGER NOT NULL CHECK (noches >= 1),   -- obligatorio y ≥ 1
  precio_noche REAL    NOT NULL CHECK (precio_noche > 0)
);
```

Restricción	Evita…
`PRIMARY KEY`	filas duplicadas
`REFERENCES`	apuntar a algo que no existe
`NOT NULL`	campos vacíos
`CHECK`	valores absurdos (noches = −3)

Índices: son como el **índice de un libro**. Sin índice la base de datos lee **todas** las filas para encontrar una (`SCAN`); con índice va directa (`SEARCH … USING INDEX`). Se comprueba con:

```
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM reservas WHERE id_cliente = 100;
```

Administración mínima: hacer copias de seguridad, controlar permisos y **no tocar producción** sin necesidad.

Errores frecuentes en SQL

Síntoma	Causa probable	Arreglo
`syntax error`	Falta una coma o el punto y coma	Revisa línea a línea
`no such column`	Nombre mal escrito o falta el alias (`r.`)	Comprueba columnas con `SELECT * … LIMIT 5`
Salen filas «de más» tras un JOIN	`ON` mal escrito (cruza todo con todo)	Revisa que une la clave foránea con su primaria
`UPDATE` cambió demasiado	Faltó el `WHERE`	`ROLLBACK` si estás en transacción; practica en la copia
Texto sin resultados	Comillas dobles o mayúsculas distintas	Usa comillas simples y el texto exacto

DÍA 1 · NoSQL: cuando las filas y columnas no bastan

8. De filas relacionadas a un documento

En SQL, ver **una reserva con su alojamiento y su cliente** exige un JOIN de tres tablas. En una base **documental** (MongoDB) se guarda **todo junto** en un documento:

```
{"_id": 5760, "canal": "empresa", "noches": 3, "precio_noche": 89.24,
 "cancelada": 1, "alojamiento": {"nombre": "…", "isla": "Tenerife"}}
```

SQL	MongoDB
Tabla	**Colección**
Fila	**Documento** (parecido a un diccionario)
Esquema fijo	**Sin esquema fijo**
Base de datos = conjunto de tablas	Base de datos = conjunto de colecciones

En el cuaderno usamos **mongomock**: MongoDB «de mentira» en memoria, para practicar sin instalar nada.

9. ¿SQL o NoSQL? No es «uno u otro»

	Relacional (SQL)	Documental (NoSQL)
Estructura	Tablas fijas con relaciones	Documentos flexibles
Mejor para	Reservas, facturas, **cifras que deben cuadrar**	Reseñas, navegación, catálogos con **atributos distintos**
Cambiar la estructura	Hay que migrar	Basta añadir un campo
Punto débil	Rigidez	**Duplicación**: si el alojamiento cambia de nombre hay que actualizar muchos documentos

Muchos proyectos usan **las dos**. La pregunta es: *¿qué tipo de dato tengo y qué voy a hacer con él?*

10. Consultar con `find`

La misma pregunta de SQL, en MongoDB:

SQL	MongoDB
`WHERE canal = 'directo'`	`{"canal": "directo"}`
`WHERE noches > 5`	`{"noches": {"$gt": 5}}`
`WHERE canal IN (…)`	`{"canal": {"$in": [...]}}`
`UPDATE … SET`	`update_one(filtro, {"$set": {...}})`
`GROUP BY`	`aggregate([{"$match": …}, {"$group": …}])`

Los símbolos con `$` son **operadores**: `$gt` = *greater than* (mayor que), `$in` = *dentro de*.

```
n = coleccion.count_documents({"cancelada": True})
caras = list(coleccion.find({"alojamiento.isla": "Tenerife", "precio_noche": {"$gt": 150}}))
```

`alojamiento.isla` es la forma de llegar **dentro** de un documento anidado.

11. Agregaciones: el GROUP BY de MongoDB

Una **pipeline** es una **lista de etapas**: cada una transforma el resultado de la anterior (como una cadena de montaje).

```
pipeline = [
  {"$group": {"_id": "$canal",
              "reservas":   {"$sum": 1},
              "canceladas": {"$sum": {"$cond": ["$cancelada", 1, 0]}}}},
  {"$sort": {"reservas": -1}},
]
```

Equivalencias: `$match` ≈ `WHERE` · `$group` ≈ `GROUP BY` · `$sort` / `$limit` ≈ `ORDER BY` / `LIMIT`. Y como en SQL: **sin filtro se afecta a todo**.

DÍA 2 · AutoML y MLOps: vuestro primer modelo, de principio a fin

12. Qué automatiza AutoML (y qué no)

AutoML es una herramienta que **prueba muchos modelos por ti** y te devuelve un ranking.

Lo que SÍ hace	Lo que NO hace
Prueba varios algoritmos	**Formular el problema**
Transforma variables	**Vigilar la fuga de datos**
Ajusta hiperparámetros	**Elegir el umbral** según el negocio
Ordena los modelos por la métrica que elijas	**Explicar el resultado al cliente**

Para qué sirve: da una **línea base fiable en minutos**. Pero **amplifica vuestro criterio; no lo sustituye**: si le das un problema mal planteado, te devolverá un modelo perfecto de nada.

El flujo del reto:

```
1. Datos → mirar el CSV con SQL y decidir qué predecir: «cancelada»
2. Revisar las columnas: ¿se conocen al reservar? (fuga de datos)
3. Experimento AutoML (AutoGluon) → ranking de modelos
4. Umbral elegido por dinero, no por costumbre
5. Servicio por API → una llamada al servicio
6. Cerrar y borrar recursos → informe de coste 0 €
```

13. Fuga de datos y partición temporal

Fuga de datos (data leakage): usar información que **no se conoce en el momento de decidir**. El modelo hace trampa y sus resultados parecen mejores de lo que son.

Ejemplo: tienes una columna `fecha_cancelacion`. Solo tiene valor **si la reserva ya se canceló**. Si la dejas, el modelo «predice» perfecto: mira si hay fecha de cancelación y listo. Pero cuando queramos usarlo para una reserva **futura**, esa columna **no existirá todavía**. (En el CSV de TurisData **no existe**: es solo un ejemplo de lo que sería fuga. Aun así, hay que revisar cada columna y razonarlo.)

Pregunta de control para cada columna: *¿este dato lo tendría yo en el momento de hacer la predicción?* Si no, se elimina.

Partición temporal: se entrena con las reservas **antiguas** y se evalúa con las **recientes**, como pasará en la realidad.

> El modelo que mejor puntúa **con fuga** no vale nada en producción.

14. Elegir el umbral con criterio de negocio

El modelo no dice «cancela» o «no cancela»: da una **probabilidad** (por ejemplo, 70 %). Tú decides **a partir de qué valor das la alerta**: ese es el **umbral**. Si el umbral es 60 % y el modelo dice 70 %, se da la alerta.

Los dos errores no cuestan lo mismo:

Error	Qué pasa	Coste
Cancelación no prevista	No avisamos y la habitación queda vacía	**Alto**
Alerta falsa	Avisamos (por ejemplo, ofrecemos un descuento) y la reserva se mantiene	**Menor**

Ejemplo numérico completo

Supón que una cancelación no prevista cuesta **100 €** y una alerta falsa **10 €**. El modelo da estas probabilidades para 8 reservas (y sabemos qué pasó de verdad):

Reserva	A	B	C	D	E	F	G	H
Probabilidad	0,90	0,80	0,70	0,60	0,45	0,35	0,20	0,10
¿Canceló?	sí	sí	no	sí	sí	no	no	no

Umbral 0,5 (el «de siempre»): alertamos a A, B, C, D.

```
Aciertos de cancelación (VP): A, B, D = 3
Alertas falsas (FP): C = 1          Canceladas que se nos escapan (FN): E = 1
Coste = 1 × 100 € + 1 × 10 € = 110 €
```

Umbral 0,3: alertamos a A, B, C, D, E, F.

```
VP = 4 (A, B, D, E)    FP = 2 (C, F)    FN = 0
Coste = 0 × 100 € + 2 × 10 € = 20 €
```

Umbral 0,1: alertamos a las 8 → FP = 4 → coste = 4 × 10 € = 40 €.

Umbral	Coste total
0,5	110 €
0,3	**20 €** ← el mejor
0,1	40 €

Conclusión: el umbral por defecto (0,5) **no es sagrado**; se elige el que **menos cuesta** según el negocio. Esto se verá con más detalle en el Sprint 13.

15. MLOps: de entrenar a operar

Un modelo que vive en un cuaderno **no sirve a nadie**. **MLOps** es el conjunto de prácticas para **poner un modelo a funcionar y mantenerlo**.

- **Servicio (API):** una **puerta** por la que otra aplicación pregunta al modelo y recibe la respuesta (probabilidad y alerta). Imagina la ventanilla de una oficina: tú no entras a las máquinas, solo pasas un papel y te devuelven la respuesta.
- **Ciclo de vida:**

```
datos → entrenar → evaluar → publicar → vigilar → reentrenar
   ↑                                                  |
   └──────────────────────────────────────────────────┘
```

- **Recursos en la nube:** todo lo que creáis (datos, experimentos, servicios activos) **consume cuota o dinero**.

> **Regla del reto:** al terminar, **cerráis y borráis todo lo que habéis creado** y dejáis prueba en el **informe de recursos** (coste 0 €). Un servicio que se queda encendido por olvido puede generar gasto.

---

16. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
SELECT, WHERE, ORDER BY	2	Script `01_select_where.sql`
JOIN y LEFT JOIN	3	`02_join.sql`
GROUP BY, HAVING	4	`03_group_by.sql`
Subconsultas y CTE	5	`04_subconsultas.sql`
INSERT, UPDATE, DELETE, ROLLBACK	6	`05_crud.sql` (en la copia)
Restricciones e índices	7	`06_modelado_indices.sql`
Documentos, `find`, `aggregate`	8-11	Cuaderno `S03_01_nosql_mongomock`
Fuga de datos, umbral, API	12-15	Guía de AutoML y reto «Primer modelo en producción»

17. Chuleta de vocabulario

Palabra	En lengua normal
Tabla / fila / columna	Una «hoja», un registro y un campo
Clave primaria	Identificador único de cada fila
Clave foránea	Columna que apunta a la clave primaria de otra tabla
JOIN	Unir dos tablas por una columna común
NULL	Vacío (no es 0 ni texto vacío)
GROUP BY	Agrupar filas y resumirlas
Subconsulta	Una consulta dentro de otra
CTE (`WITH`)	Ponerle nombre a un paso intermedio
CRUD	Crear, leer, actualizar y borrar
Transacción	Grupo de cambios que se puede deshacer (`ROLLBACK`)
Índice	«Índice de libro» que acelera búsquedas
Documento / colección	Fila / tabla en MongoDB
AutoML	Herramienta que prueba modelos por ti
Fuga de datos	El modelo ve información que no tendría al decidir
Umbral	Probabilidad a partir de la cual se da la alerta

Palabra	En lengua normal
API	Puerta para que otras aplicaciones usen el modelo
MLOps	Poner el modelo en marcha y mantenerlo

18. Cómo estudiar estos apuntes

1. **Antes de los scripts SQL**, lee las secciones 1-7 con la mini-base delante y **comprueba a mano** un par de resultados.
2. **Si una consulta falla**, mira la tabla de errores frecuentes (sección 7).
3. **Antes del reto**, relee las secciones 12-15, sobre todo el **ejemplo numérico del umbral**: es lo que debéis saber explicar al cliente.
4. **No memorices la sintaxis**: la práctica la fija y puedes consultar los scripts.